

SK네트웍스 Family AI과정 21기

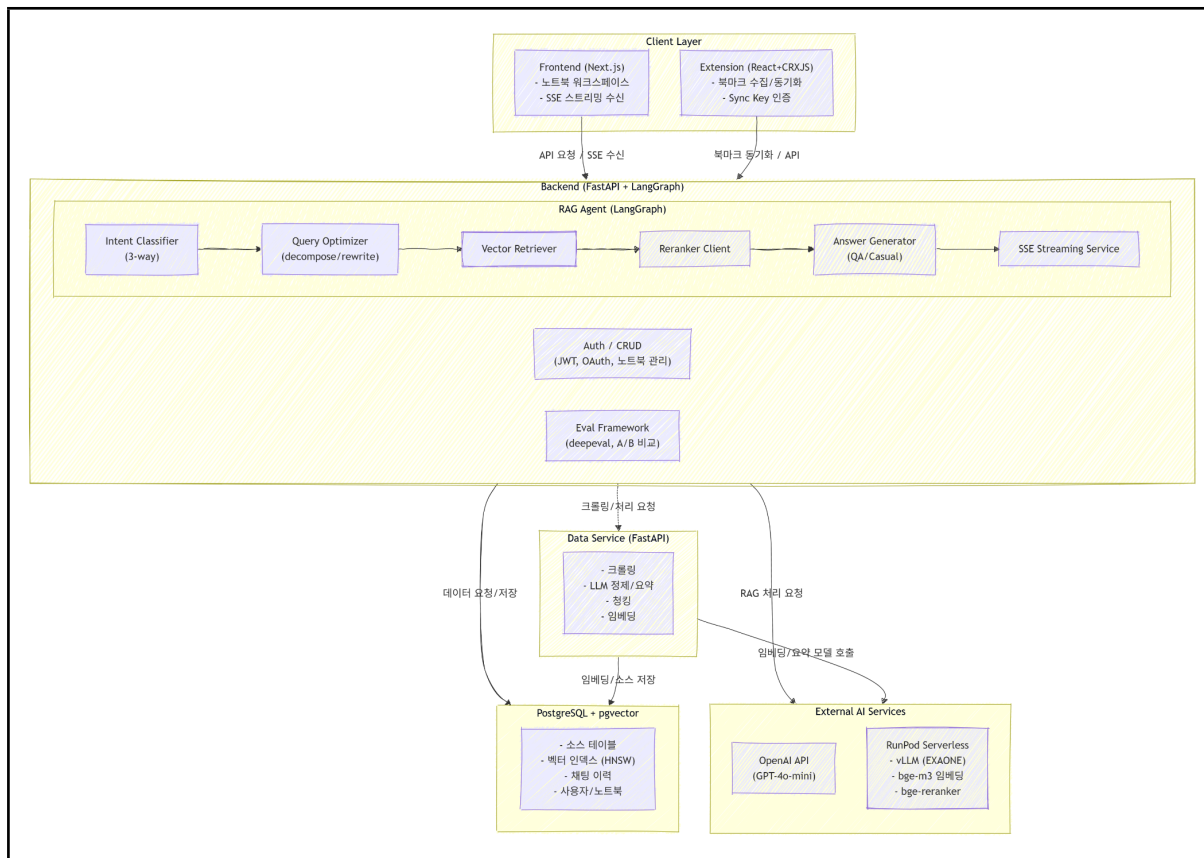
모델링 및 평가 시스템 아키텍처

□ 개요

- 산출물 단계 : 모델링 및 평가
- 평가 산출물 : 시스템 아키텍처
- 제출 일자 : 2026. 03. 24.
- 깃허브 경로 : <https://github.com/skn21-final-1team>
- 작성 팀원 : 우재현

1. 컴포넌트 다이어그램

구성 요소



설명

구성 요소	기술 스택	역할
Frontend	Next.js 16, Zustand, Axios	노트북 UI, SSE 스트리밍 수신, 소스 인용 호버 표시
Chrome Extension	React, CRXJS (Manifest V3)	브라우저 북마크 수집, Sync Key 기반 Kalpie 동기화
Backend	FastAPI, LangGraph	RAG 에이전트 오케스트레이션, 인증, API 라우팅
RAG Agent	LangGraph (StateGraph)	의도 분류 → 쿼리 최적화 → 벡터 검색 → 리랭킹 → 답변 생성
Eval Framework	deepeval, MLflow	LLM-as-Judge 5개 메트릭 평가, A/B 비교, 배치 평가
Data Service	FastAPI, Trafilatura, Playwright	웹 크롤링 → LLM 정제/요약 → 청킹 → 임베딩 → DB 적재
PostgreSQL + pgvector	PostgreSQL, pgvector (HNSW)	소스/청크/벡터 저장, 벡터 유사도 검색
OpenAI API	GPT-4o-mini	답변 생성, 의도 분류, 쿼리 최적화, 평가 Judge
RunPod Serverless	vLLM, bge-m3, bge-reranker-v2-m3	EXAONE 답변 생성, 임베딩 생성, Cross-Encoder 리랭킹

- 클라이언트(Frontend/Extension)는 JWT 인증을 거쳐 Backend에 JSON 요청을 전송한다.
- Backend의 LangGraph RAG Agent는 질문 의도를 분류하고, PostgreSQL+pgvector에서 벡터 검색을 수행한 뒤 RunPod의 Cross-Encoder로 리랭킹하여 관련 소스를 선별한다.
- 선별된 소스를 기반으로 OpenAI API(GPT-4o-mini) 또는 RunPod(EXAONE)에서 답변을 생성하며, SSE 스트리밍으로 클라이언트에 실시간 전송한다.
- 데이터 수집 시에는 Backend가 Data Service에 크롤링을 요청하고, Data Service는 RunPod의 vLLM으로 정제/요약, bge-m3로 임베딩을 생성하여 PostgreSQL에 벡터와 함께 적재한다.
- deepeval 평가 프레임워크는 테스트셋 50건에 대해 RAG 파이프라인을 실행하고, GPT-4o-mini를 Judge로 사용하여 5개 메트릭을 산출한다.

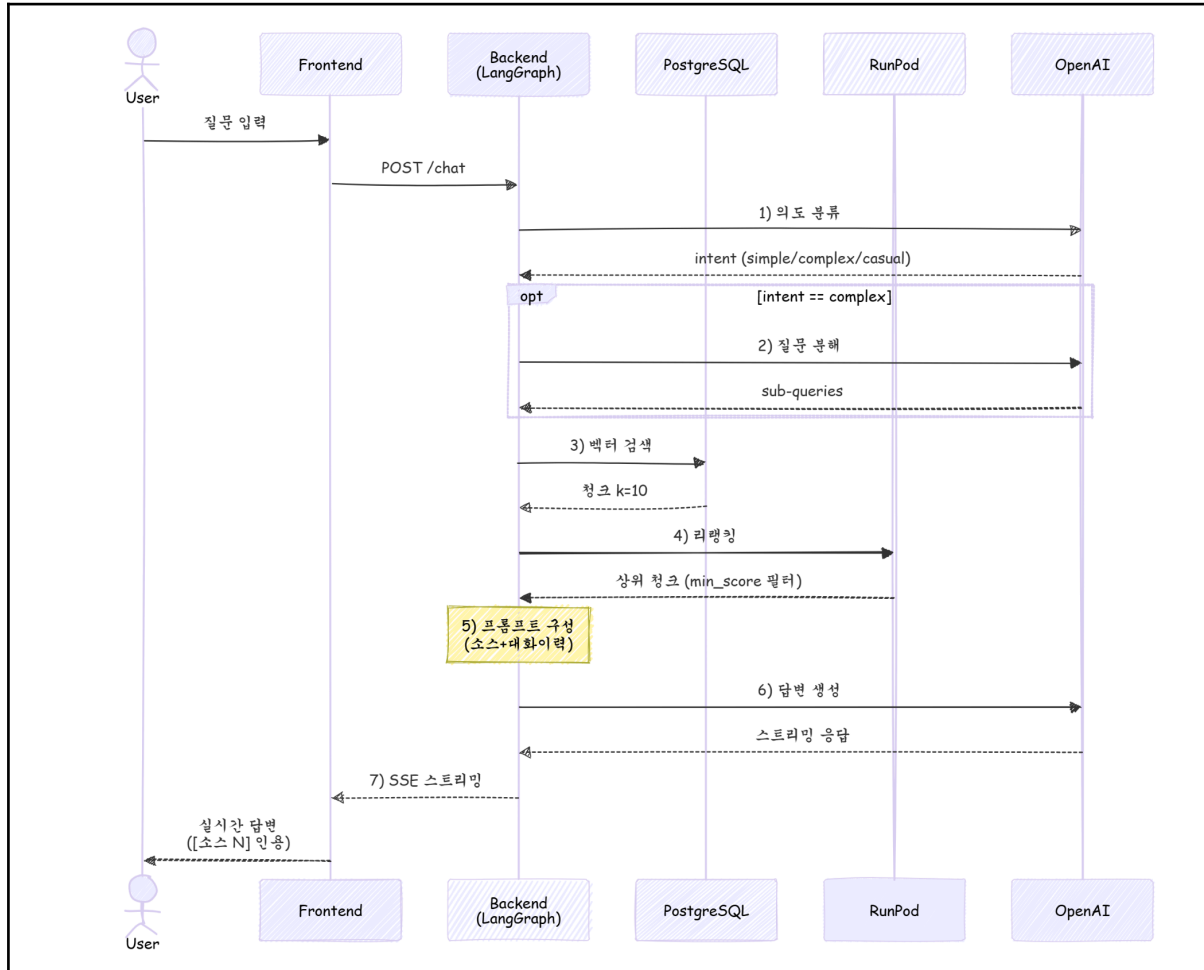
2. 시퀀스 다이어그램

참여자 (Actors)

참여자	역할
User	질문 입력, 소스 관리
Frontend	UI 렌더링, SSE 스트리밍 수신
Backend	RAG 에이전트 실행, API 라우팅
PostgreSQL	벡터 검색, 데이터 저장
RunPod	리랭킹, 임베딩, EXAONE 답변 생성
OpenAI	GPT-4o-mini 답변 생성
Data Service	크롤링, 정제, 임베딩 파이프라인

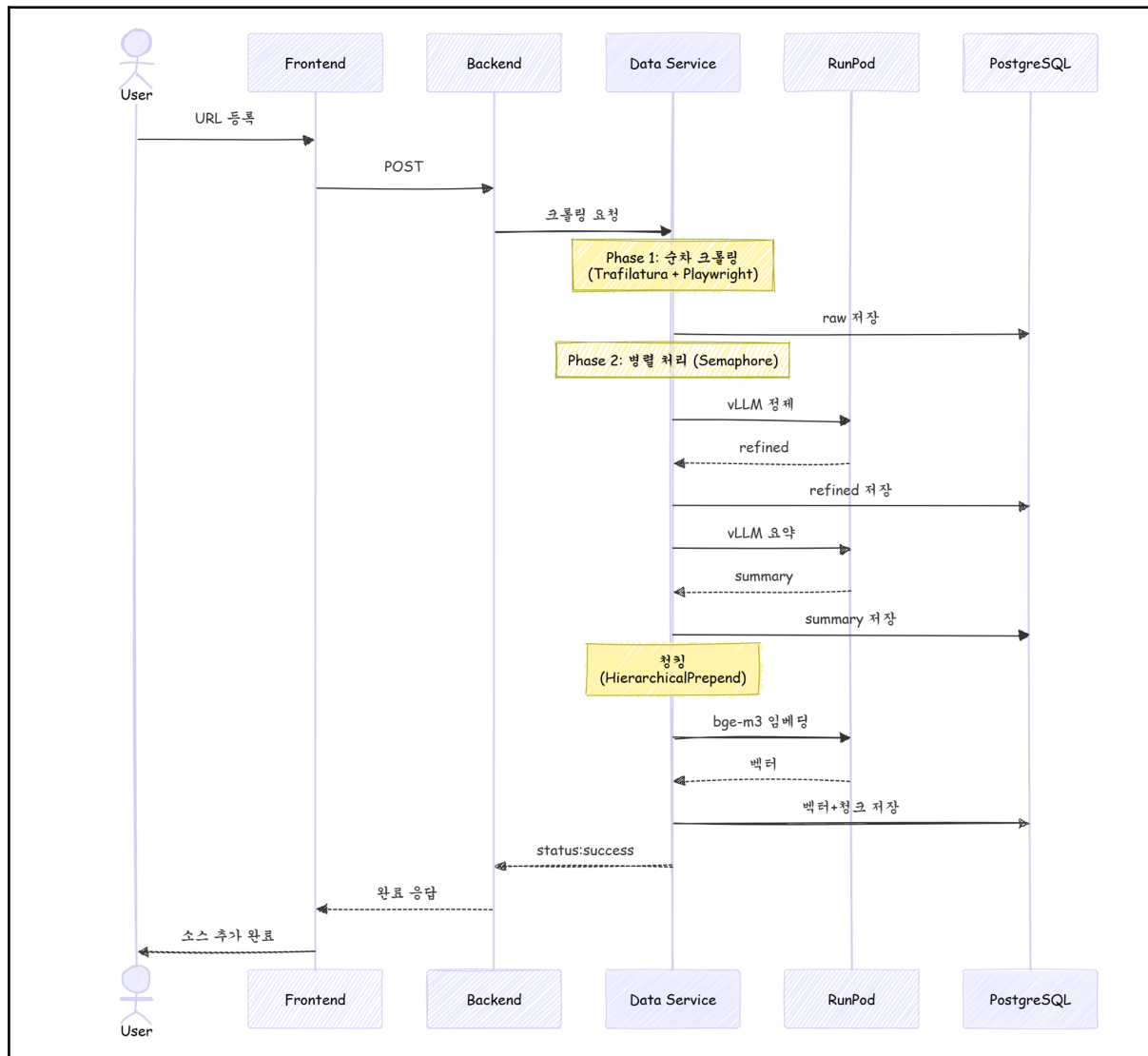
주요 흐름

흐름 1: RAG 답변 생성



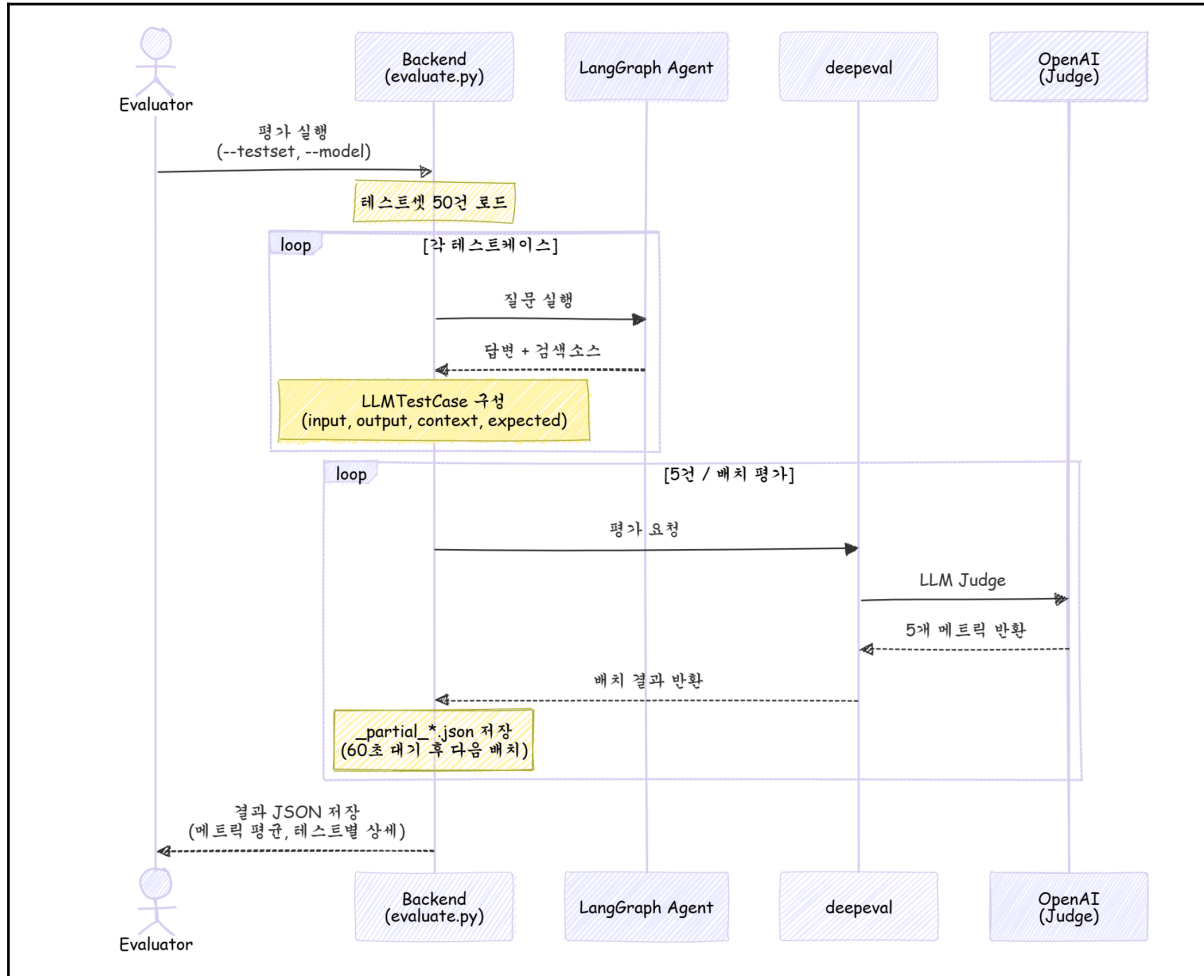
- 사용자가 질문을 입력하면 Frontend가 Backend에 POST 요청을 전송하고, LangGraph RAG Agent가 7단계 파이프라인을 실행한다.
- 의도 분류(OpenAI)에서 simple/complex/casual을 판별하고, complex일 경우 질문 분해와 쿼리 재작성을 추가 수행한다.
- PostgreSQL에서 벡터 검색(k=10)으로 후보 청크를 가져온 뒤, RunPod의 bge-reranker로 관련성 순 재정렬하고 min_score 미만을 필터링한다.
- 최종 소스와 대화 이력을 조합하여 프롬프트를 구성하고, OpenAI(GPT-4o-mini)에서 답변을 생성하여 SSE로 실시간 스트리밍한다.

흐름 2: 데이터 수집 파이프라인



- 사용자가 URL을 등록하면 Backend를 거쳐 Data Service로 크롤링 요청이 전달된다.
- Phase 1에서 Trafilatura(정적) + Playwright(동적 폴백)로 순차 크롤링하여 원본(raw)을 DB에 저장한다.
- Phase 2에서 asyncio.Semaphore로 병렬 처리하며, RunPod의 vLLM으로 정제/요약하고 bge-m3로 임베딩을 생성한다.
- 각 단계 결과를 DB에 저장(체크포인트)하여 실패 시 완료된 단계부터 재시도할 수 있다.

흐름 3: 평가 실행



- evaluate.py가 테스트셋 50건을 로드하고, 각 질문에 대해 LangGraph RAG Agent를 실행하여 답변과 검색 소스를 수집한다.
- 수집된 결과를 LLMTestCase로 구성하여 deepeval에 5건/배치로 전달하며, GPT-4o-mini가 Judge로서 5개 메트릭을 채점한다.
- 배치 간 60초 대기를 두어 API Rate Limit을 방지하고, 중간 결과를 `\_partial\_\*.json`에 저장하여 크래시 시 `--resume`으로 이어서 평가한다.

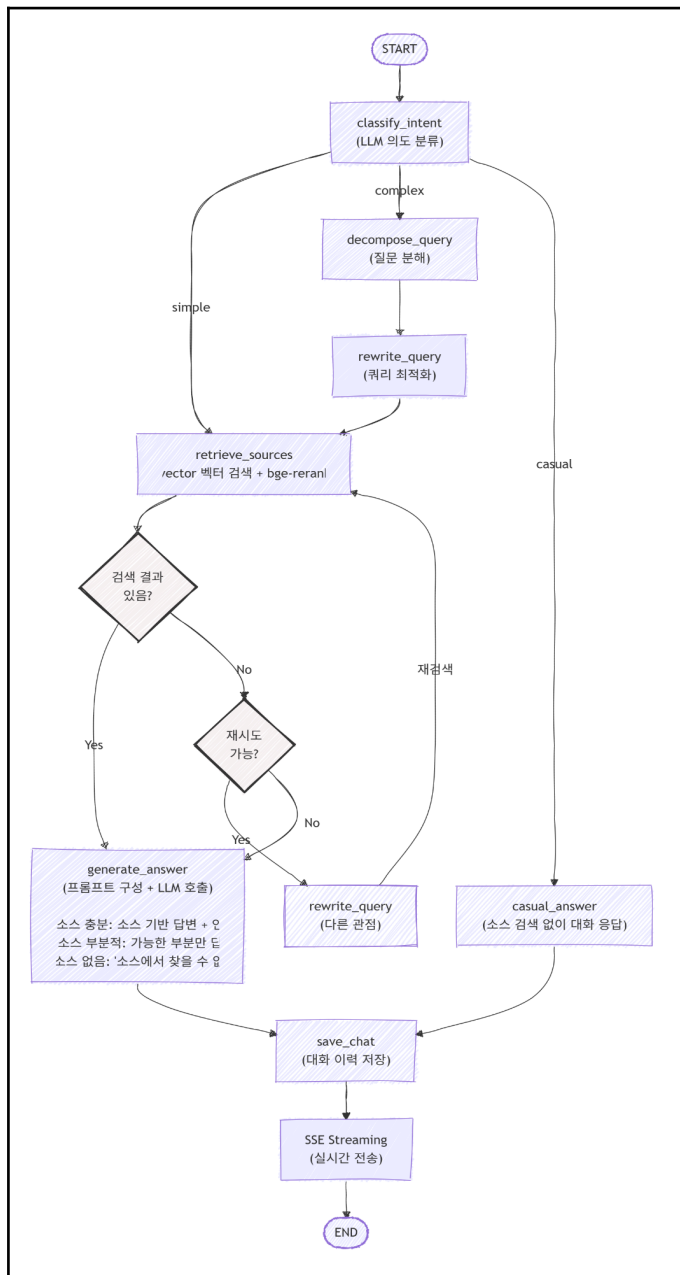
3. 액티비티 다이어그램

구성

[시작] → 의도 분류 → <분기> → 검색/생성 → [종료]
 |
 simple / complex / casual

주요 액션 노드

RAG 파이프라인 액티비티

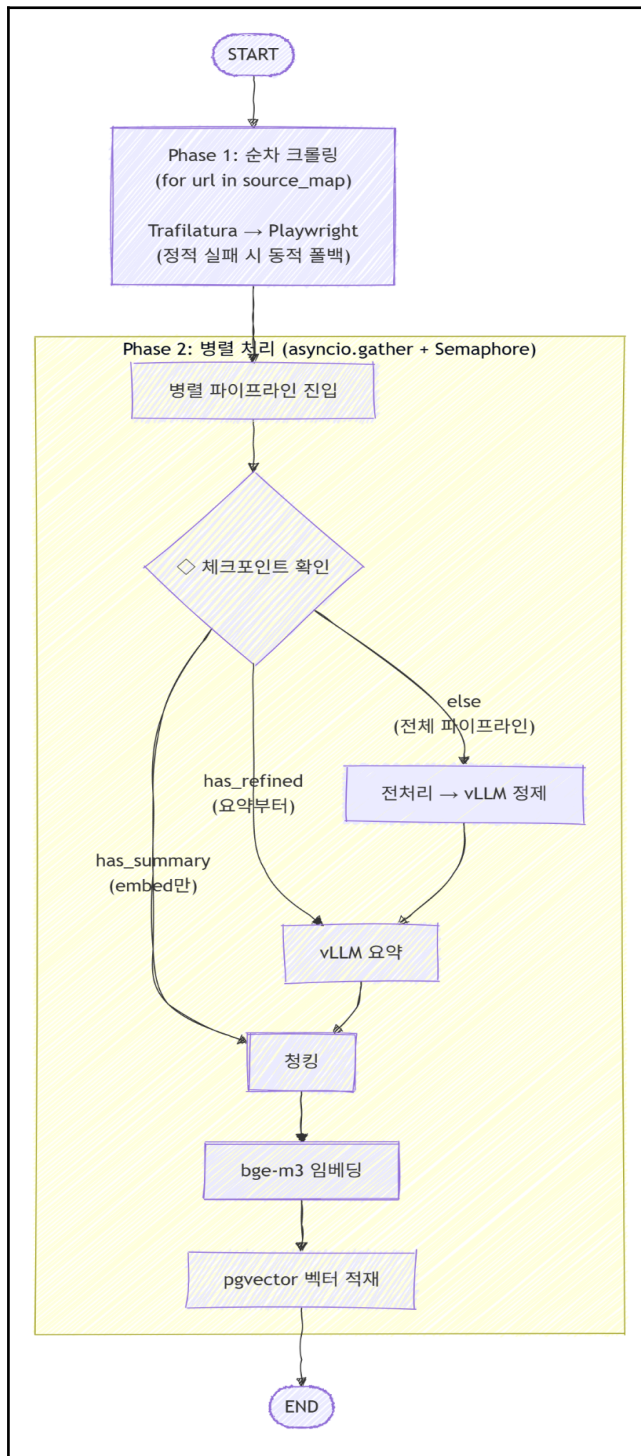


- RAG 파이프라인은 의도 분류에서 **simple/complex/casual** 3갈래로 분기하며, **casual**은 소스 검색 없이 바로 대화 응답을 생성한다.
- **simple** 질문은 **rewrite_query**를 건너뛰고 바로 벡터 검색을 실행하여 불필요한 LLM 호출을 제거한다.
- **complex** 질문은 **sub-query** 분해 → 쿼리 재작성 → 벡터 검색 → 리랭킹의 전체 경로를 거친다.
- 검색 결과가 0건이면 다른 관점으로 쿼리를 재작성하여 1회 재시도하고, 그래도 없으면 **Graceful Degradation**으로 "소스에서 찾을 수 없음"을 안내한다.

액션 노드 설명

액션 노드	처리 내용	분기 조건	파일
classify_intent	LLM으로 질문 의도를 simple/complex/casual 3-way 분류	`route_by_intent`: intent 값에 따라 3갈래 분기	`nodes/classify_intent.py`
decompose_query	complex 질문을 2~3개 sub-query로 분해	complex일 때만 실행	`nodes/decompose_query.py`
rewrite_query	대명사 복원, 키워드 추출로 검색 쿼리 최적화	complex일 때만 실행 (simple은 스킵)	`nodes/rewrite_query.py`
retrieve_sources	pgvector 벡터 검색(k=10) + bge-reranker 리랭킹	-	`nodes/retrieve_sources.py`
(재시도 분기)	검색 결과 0건 시 쿼리 재작성 후 재검색 (최대 1회)	`route_by_sources`: 결과 유무 + retrieval_count	`edges/route_by_sources.py`
generate_answer	소스 + 대화 이력으로 프롬프트 구성, LLM 호출	소스 충분/부분적/없음 3단계 대응	`nodes/generate_answer.py`
casual_answer	소스 검색 없이 자연스러운 대화 응답	casual일 때만 실행	`nodes/generate_answer.py`
save_chat	질문-답변 쌍을 DB에 저장	-	`nodes/save_chat.py`
SSE Streaming	LangGraph astream()으로 실시간 토큰 전송	-	`services/agent.py`

데이터 수집 파이프라인 액티비티



- 데이터 수집은 **Phase 1**(순차 크롤링)과 **Phase 2**(병렬 LLM+임베딩) 2단계로 구성된다.
- **Phase 1**에서 외부 사이트 부하 방지를 위해 순차 실행하고, **Phase 2**에서 **Semaphore**로 동시 처리 수를 제한하며 병렬 처리한다.
- 재시도 시 **DB** 체크포인트를 확인하여 이미 완료된 단계(정제/요약)는 건너뛰고 남은 단계부터 실행한다.